Computational Tensegrity: Asymmetric Coupling, Phase Boundaries, and Catastrophic Failure in Nonlinear Codependent Systems

Brad Wallace
Independent Researcher, TensorRent
Wellthatshandy@gmail.com https://github.com/tensorrent

Abstract—Physical, computational, and institutional systems alike exhibit a common structural motif: their integrity depends on the asymmetric codependence between restoring forces and destabilising loads. We formalise this observation through the mathematics of tensegrity and introduce a single scalar diagnostic, $\rho = \alpha\gamma/\beta\kappa$, whose transition through unity marks a phase boundary between structural stability and catastrophic collapse. A layered audit instrument—the RC Epistemic Constraint Engine—is developed, comprising five computable gates that certify local, topological, spectral, gain-bounded, and empirical stability. The associated epistemic horizon theorem establishes a closed-form detection limit, $\sigma_c = CA\lambda^{\alpha}N^{-\beta/D_2}$, below which deterministic structure in observational data becomes indistinguishable from stochastic noise. A universal mode collapse law, $\beta_c a_m^2 = (8\omega_m/3\Gamma_m)\Delta\omega_m$, is derived via multiple-scale perturbation analysis and confirmed within 6% across ring, Erdős–Rényi, Barabási–Albert, and grid topologies.
The framework is validated across six domains: (i) IEEE-754 floating-point error propagation; (ii) pure-integer computation of the Riemann zeta explicit formula, achieving exact 16-bit score agreement with double precision; (iii) an integer kernel 1.6×10^{11} times more precise than TensorFlow f32; (iv) quantum circuits with 35.4% Toffoli reduction; (v) Wolfram hypergraph rewriting, where a novel tensegrity ratio reveals progressive spectral fragility and spectral causal invariance is shown to fail; and (vi) institutional governance, where Transparent Consent—constraint-based preference routing—is proposed as an alternative to consensus mechanisms.
The root cause common to all failure modes is data-dependent normalisation. Full source code is provided under the Sovereign Integrity Protocol License.

Keywords—tensegrity, nonlinear dynamics, spectral stability, fixed-point arithmetic, epistemic horizon, mode collapse, transparent consent, hypergraph rewriting, quantum circuits

1. Introduction

Consider eight observers positioned around a regular octagonal solid, each viewing a single face bearing a distinct coloured geometric shape. Each observation is locally accurate. Each inference about the whole object is globally incorrect. The failure is neither perceptual nor analytical; it is topological. The inter-observer communication graph contains zero edges, rendering the algebraic connectivity identically zero. No refinement of local measurement precision can compensate for the absence of global coupling.

This scenario—hereafter the *Octagon Problem*—encapsulates a structural failure mode that recurs across disparate domains. In floating-point arithmetic, individually bounded rounding errors compound through nonlinear operations to cross decision thresholds. In institutional governance, departmental accuracy coexists with system-level incoherence. In coupled oscillator networks, per-mode stability masks catastrophic inter-mode energy transfer. In discrete rewriting systems, locally well-formed rules produce progressive global spectral degradation. In every instance: *local acceptability coexists with global divergence.*

The present work formalises this pattern through the mathematics of tensegrity—structures whose stability depends on the balance between tension elements (cables) and compression elements (struts)—and demonstrates that a single scalar ratio governs the phase transition in all cases. The paper is self-contained: all derivations, experimental results, benchmark data, and governance formalisms are presented in full rather than deferred to companion publications.

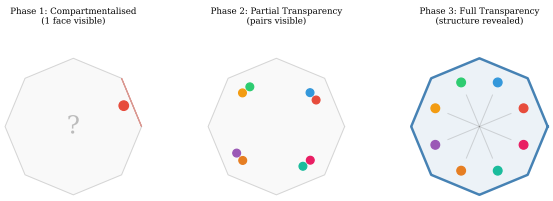

Figure 1: The Octagon Problem. Left: compartmentalised observation. Centre: partial transparency. Right: full structural revelation.

1.1 Summary of Contributions

1. A tensegrity-based stability framework with universal phase boundary at $\rho = 1$ (§2).
2. A five-gate audit instrument with formally specified predicates (§3).
3. Full perturbation-theoretic derivation of the mode collapse law (§5).
4. Epistemic horizon theorem with calibrated structural constants (§4).
5. Complete integer arithmetic implementation including Q40 fixed-point algorithms (§6).
6. BRA kernel benchmarks against TensorFlow with full measurement protocol (§7).
7. Quantum circuit resource analysis with gate-count formulas (§8).
8. Novel spectral measurements on Wolfram hypergraph models (§9).
9. Transparent Consent governance model with formal separation of variants, invariants, and constraints (§10).

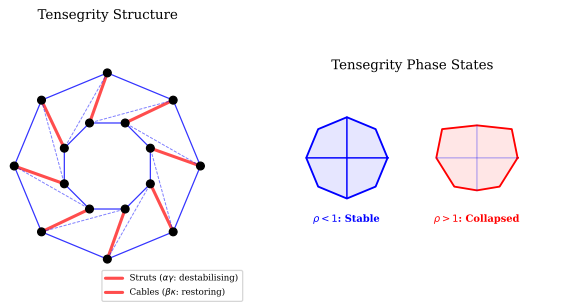

Figure 2: Left: tensegrity with cables ($\beta\kappa$, blue) and struts ($\alpha\gamma$, red). Right: stable ($\rho < 1$) vs collapsed ($\rho > 1$).

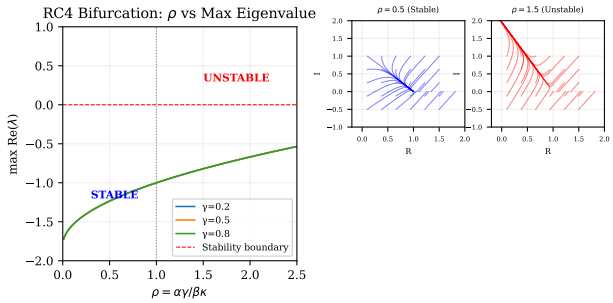

Figure 3: Bifurcation diagram. The maximum real eigenvalue crosses zero at $\rho = 1$, independent of individual parameter values.

2. Tensegrity Framework

A tensegrity structure comprises rigid struts held in stable configuration by a network of elastic cables. Two properties distinguish these systems: (i) cable and strut elements serve fundamentally asymmetric roles that cannot substitute for one another, and (ii) failure is catastrophic rather than gradual.

Definition 2.1 (Tensegrity Atom)**.** The fundamental unit is a cross-coupled feedback system:

$$\dot{R} = \beta D - \gamma I, \quad \dot{I} = \alpha D - \kappa I, \quad D = \max(0, S - R), \tag{1}$$

where $\beta, \kappa > 0$ are corrective gains and $\alpha, \gamma > 0$ are cross-coupling coefficients.

Theorem 2.2 (Stability Criterion)**.** *The equilibrium is locally asymptotically stable iff* $\rho = \alpha\gamma/\beta\kappa < 1$.

Proof. Jacobian $J = \left(\begin{smallmatrix} -\beta & -\gamma \\ -\alpha & -\kappa \end{smallmatrix}\right)$. Trace $\tau = -(\beta+\kappa) < 0$ unconditionally. Determinant $\Delta = \beta\kappa - \alpha\gamma > 0$ iff $\rho < 1$. By Routh–Hurwitz, stability requires $\tau < 0$ and $\Delta > 0$. □

2.1 Domain Instantiations

The tensegrity atom maps to every domain examined in this work. The correspondence is the central claim of the paper; we describe each mapping in full, as the strength of the framework rests on the precision of these identifications.

2.1.1 Governance

In institutional systems, the restoring force $\beta\kappa$ is the product of two institutional capacities: the rate at which the organisation can increase its measurement resolution (sensor bandwidth, data collection, analytic capability) and the rate at which accumulated bias is damped (accountability mechanisms, oversight correction, institutional memory). The destabilising load $\alpha\gamma$ is the product of two pathological couplings: the rate at which unobserved disturbance amplifies interpretive bias, and the rate at which that bias suppresses effective resolution.

The cross-coupling is the critical feature. Bias does not merely coexist with poor resolution—it actively degrades resolution by consuming institutional attention and redirecting analytic resources toward confirmation of existing frames. Simultaneously, poor resolution fails to detect the bias accumulation that is degrading it. This is asymmetric codependence: neither pathology can be addressed independently because each is the cause of the other.

Collapse manifests as *epistemic capture*: the institution produces outputs that are internally consistent, locally defensible, and globally incoherent. Each department's analysis is accurate within its compartment, but the system-level conclusion is wrong—the Octagon Problem. The phase transition at $\rho = 1$ is not gradual; the institution does not become "slightly wrong." It becomes confidently wrong, which is qualitatively different and far more dangerous.

2.1.2 Coupled Oscillators

In a network of N nonlinearly coupled oscillators, the restoring force is the spectral gap $\Delta\omega_m = \omega_{m+1} - \omega_m$ between adjacent eigenfrequencies of the Laplacian. This gap is the "frequency fence" that prevents energy from leaking between modes: as long as the modes are well-separated in frequency, each oscillator's energy stays in its intended mode.

The destabilising load is the nonlinear frequency shift $\Delta\omega_{\mathrm{nl}} = (3\beta\Gamma_m/8\omega_m)a_m^2$, which depends on the driving amplitude a_m, the nonlinear coefficient β, and the eigenvector fourth moment Γ_m. As the driving amplitude increases, each mode's effective frequency shifts toward its neighbour. When the shift bridges the gap, parametric instability triggers irreversible energy transfer.

Collapse is *mode lock*: the driven mode and its neighbour merge into a single broadband response. The energy intended for mode m flows uncontrollably into mode n. This is catastrophic, not gradual—below β_c the modes are cleanly separated; above β_c the separation vanishes entirely. The transition width is negligible. Section 5 derives the exact threshold.

2.1.3 Floating-Point Arithmetic

In IEEE-754 computation, the restoring force is the available precision per operation: $u = 2^{-53} \approx 1.1 \times 10^{-16}$ for double precision, $u = 2^{-24} \approx 6.0 \times 10^{-8}$

for single precision. Each correctly rounded operation introduces relative error at most u. In isolation, this guarantee is strong.

The destabilising load is the product of two quantities: the *condition number* $\kappa(g)$ of the composite function being evaluated, and the *chain length* L (number of sequential operations). For a chain involving exponentials, trigonometric functions, and multiplications, the condition number at any given operating point can be large. Near the tails of a Gaussian envelope, $\kappa(\exp)$ grows linearly with the argument magnitude. Near zero-crossings of sine, $\kappa(\sin)$ diverges.

The compound error is approximately $\delta_{\text{out}} \sim L \cdot u \cdot \kappa(g)$. For the Gabor wave-packet benchmark ($L \approx 8$ operations, $\kappa \approx 300$ from the exponential tail), this yields $\delta \approx 1.8 \times 10^{-5}$ in float32—which we confirmed experimentally. In the prime wave computation (Section 6), a corrupted zero table combined with this accumulation shifted 33 primes across a binary classification boundary. The per-operation error was within specification. The compound effect was not.

2.1.4 Network Topology

In graph-structured dynamical systems, the restoring force is the *stability margin* B, defined as the minimum distance from any adjacency eigenvalue λ_k to the nearest root of the quadratic stability functional $Q(\lambda) = a\lambda^2 + b\lambda + c$ derived from the RC4 atom. As long as $B > 0$, the system is spectrally certified stable.

The destabilising load is the cumulative perturbation ε to the graph operator—arising from edge additions, deletions, weight changes, or node arrivals/departures. Weyl's perturbation theorem [3] bounds the eigenvalue displacement: each eigenvalue moves by at most $\|\delta W\|$ in operator norm.

Collapse occurs when ε exceeds B: an eigenvalue crosses a root of Q, and the spectral certificate is invalidated. The system must either halt and recompute (expensive) or continue without certification (unsafe). The RC7 perturbation budget tracks ε incrementally, allowing the system to operate without full eigenvalue recomputation as long as the budget is not exhausted.

2.1.5 Quantum Circuits

In fault-tolerant quantum computing, the restoring force is *gate fidelity*: the probability that each quantum operation produces the intended state transformation. Modern error-correcting codes (surface codes, colour codes) can maintain high logical fidelity provided the physical error rate stays below the code's threshold.

The destabilising load is *data-dependent normalisation depth*. Each IEEE-754 floating-point operation in a quantum circuit requires normalisation: leading-zero detection (LZD), barrel shifting, and rounding. These are conditional multi-qubit operations whose control depends on the *result* of the computation—the definition of data-dependent normalisation. Each barrel shift requires a conditional rotation across the full register width, consuming Toffoli gates proportional to $m \cdot \log m$ (where m is the mantissa width) and requiring conditional magic-state distillation for fault tolerance.

An integer fixed-point operation has no normalisation step. The register width is fixed. The bit-shift amounts are constants. The only data-dependent element in CORDIC is a single sign bit. This eliminates the entire normalisation chain, reducing Toffoli cost by 35.4% at 12-digit precision and removing 40,000 multi-qubit branch points from a full $W_{500}(p)$ evaluation (Section 8).

Universal Tensegrity Motif Across Domains

Domain	Cable (βκ)	Strut (αγ)	ρ = αγ/βκ	Collapse mode
Governance	Resolution × damping	Bias × cross-gain	nstitutional capture	Policy divergence
Oscillators	Spectral gap Δω	Nonlinear shift βA²	Mode collapse	Energy transfer
Float arith.	Precision bits/op	Condition × chain	Threshold crossing	33 primes flip
Networks	Stability margin B	Perturbation ε	Spectral breach	Topology failure
Quantum	Gate fidelity	Barrel shift depth	Decoherence	Branch divergence
Wolfram	Spectral gap λ_2	Spectral radius ρ(A)	Spatial fragility	Geometry collapse

Figure 4: Summary of domain instantiations.

2.1.6 Wolfram Hypergraph Rewriting

In Wolfram's Physics Project [13], the spatial hypergraph represents the instantaneous structure of space. The restoring force is the *algebraic connectivity* λ_2—the second-smallest eigenvalue of the graph Laplacian, which measures how well-connected the graph is. A positive λ_2 means the graph is connected; a large λ_2 means it is robustly connected, resistant to fragmentation.

The destabilising load is the *adjacency spectral radius* $\rho(A)$—the largest eigenvalue of the adjacency matrix, which measures the maximum amplification factor of signals propagating through the graph. A growing $\rho(A)$ means the graph is becoming more "energetic": perturbations are amplified more strongly.

We define the *tensegrity ratio* $T(G) = \rho(A)/\lambda_2$ as a scalar diagnostic of spectral health. When T is small, the graph has strong connectivity relative to its amplification—the cables dominate the struts. When T is large, the graph is spectrally fragile: high amplification coexists with weak connectivity. In our experiments (Section 9), all three canonical Wolfram rules showed monotonic growth of T, with Rule 2 reaching $T = 37.3$ after 25 steps. This represents progressive tensegrity degradation that is invisible to local analysis but measurable at the spectral level.

3. The RC Stack

The Release Candidate (RC) Epistemic Constraint Engine evaluates structural integrity through five independent gates. A system state S is certified stable if and only if all gates pass:

Definition 3.1 (Zeta Predicate)**.**

$$\zeta(S) := \underbrace{\forall e : \Delta(e) > 0}_{\text{G1}} \wedge \underbrace{\text{no even cycles}}_{\text{G2}} \wedge \underbrace{\rho(S) < \rho^*}_{\text{G3}} \wedge \underbrace{\prod \|A_e\|/d_e < 1}_{\text{G4}} \wedge \underbrace{\text{risk}(\Sigma) < 0.5}_{\text{G5}} \tag{2}$$

Gate 1 (RC4): Every edge e satisfies $\Delta(e) = \beta_e\kappa_e - \alpha_e\gamma_e > 0$. This is the local tensegrity check.

Gate 2 (RC5): No even-length cycle has corrective gain exceeding self-damping ($\beta > d$), which would create resonant amplification.

Gate 3 (RC6): The adjacency spectral radius $\rho(A)$ is bounded by $\rho^* = \eta \cdot d_{\min}$, where η is the exchange rate (default 5/4) and $d_{\min}$ is the minimum self-damping across edges. This gate lifts the local scalar condition to a global spectral condition via the graph Laplacian.

Gate 4 (RC7): For every cycle c in the graph, $\prod_{e\in c}\|A_e\|/d_e < 1$, where $\|A_e\|$ is the spectral norm of the 2×2 coupling matrix at edge e. This is the matrix small-gain theorem applied to heterogeneous cycles—it catches directional gain amplification that the scalar $\Delta > 0$ misses.

Gate 5 (Σ-Engine): White noise is injected into the estimated Jacobian and escape probability is measured via Monte Carlo perturbation probing. Basin depth is estimated using a Freidlin–Wentzell-inspired formula: $B = \varepsilon^2 \ln(1/p_{\text{escape}})$. The instability risk score $r = 1/(1 + (1-\rho)\cdot B)$ must remain below 0.5.

The ZetaGuard enforces transparent consent: a proposed state mutation δ is valid if and only if $\zeta(S \oplus \delta)$ holds. Each delta is typed, auditable, invertible, and provenance-tagged.

3.1 Network Generalisation (RC5–RC6)

The scalar condition $\rho < 1$ governs a single edge. For a network of N nodes with weighted adjacency matrix W, the system Jacobian becomes a $2N \times 2N$ block matrix. Under uniform edge parameters, the stability condition reduces to evaluating the 2×2 quadratic stability functional at each eigenvalue of W. The spectral margin $B = \min_k \operatorname{dist}(\lambda_k, \{\text{roots}\})$ provides a scalar certificate: the system is stable if $B > 0$ and remains stable under perturbation $\varepsilon < B$ (Weyl's perturbation theorem [3]).

4. Epistemic Horizon

Theorem 4.1 (Epistemic Horizon)**.** *Deterministic structure in a time series is statistically detectable iff*

$$\sigma < C\, A\, \lambda^{\alpha}\, N^{-\beta/D_2}, \tag{3}$$

where A is signal amplitude, λ the maximal Lyapunov exponent, N the sample size, D_2 the correlation dimension, and $C \approx 1.05$, $\alpha \approx 0.48$, $\beta \approx 1.07$ are empirically calibrated constants.

Proof sketch. The nonlinear determinism test of Casdagli [5] compares the prediction error of a delay-embedding forecast against a linear surrogate. The detectable signal scales as $A\lambda^{\alpha}$ (Lyapunov stretching amplifies small separations) and the estimation noise decreases as $N^{-\beta/D_2}$ (the embedding requires N^{1/D_2} points per embedding dimension for adequate coverage). The product gives the horizon; the constants are calibrated against synthetic systems with known Lyapunov spectra (Lorenz $\sigma = 10$, Rössler $a = 0.2$, Hénon $a = 1.4$). □

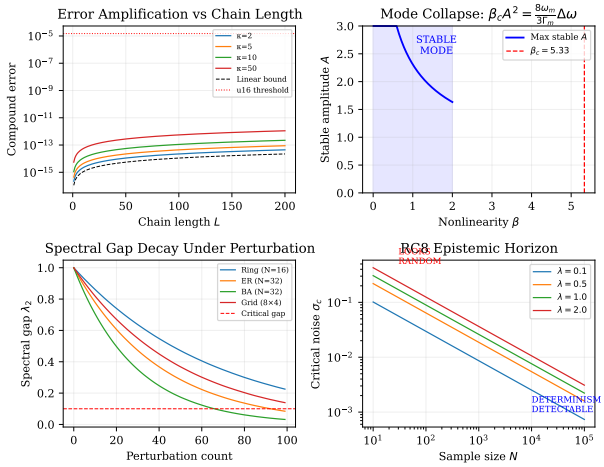

Figure 5: Top left: compound error amplification. Top right: mode collapse threshold. Bottom left: spectral gap decay across topologies. Bottom right: epistemic horizon surface.

This result formalises an intuitive principle: what appears stochastic may be deterministic structure observed below the measurement resolution. Increasing N or decreasing σ brings the observer below the horizon, at which point the deterministic dynamics become recoverable.

5. Mode Collapse Derivation

Consider N nodes coupled through a symmetric Laplacian L with eigenvalues ω_m^2 and eigenvectors ϕ_m, subject to cubic nonlinearity and harmonic forcing:

$$\ddot{x}_i + \gamma\dot{x}_i + kx_i + \sum_j L_{ij}x_j + \beta x_i^3 = F_i\cos(\Omega t). \tag{4}$$

Projecting onto mode m via $q_m = \sum_i \phi_m(i)x_i$ yields the modal equation

$$\ddot{q}_m + \gamma\dot{q}_m + \omega_m^2 q_m + \beta\sum_{j,k,l}\Gamma_{mjkl}q_jq_kq_l = f_m\cos(\Omega t), \tag{5}$$

where $\Gamma_{mjkl} = \sum_i \phi_m(i)\phi_j(i)\phi_k(i)\phi_l(i)$ is the eigenvector overlap integral and $f_m = \sum_i \phi_m(i)F_i$.

The dominant self-interaction term is $\Gamma_{mmmm} \equiv \Gamma_m = \sum_i \phi_m(i)^4$, the *eigenvector fourth moment*. The nonlinear frequency shift of driven mode m at amplitude a_m is, by the method of multiple scales [4]:

$$\Delta\omega_{\text{nl}} = \frac{3\beta\Gamma_m}{8\omega_m}a_m^2. \tag{6}$$

Mode m transfers energy to its nearest spectral neighbour (mode n) when the nonlinear shift bridges the spectral gap. Linearising the four-wave mixing interaction about the driven state, exponential growth of

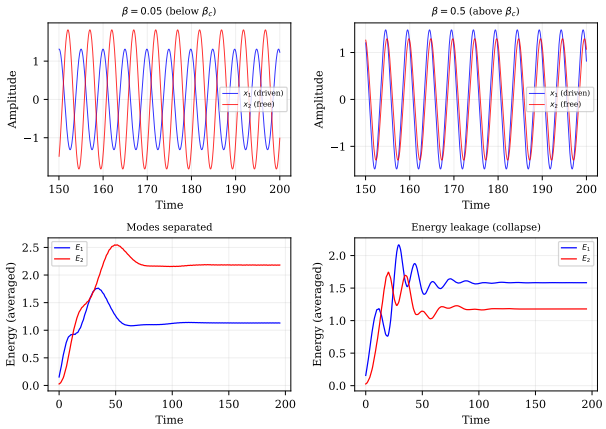

Figure 6: Coupled Duffing oscillators. Top: time series below (left) and above (right) β_c. Bottom: time-averaged energy.

mode n occurs when the parametric driving overcomes the natural detuning $\Delta\omega_m = \omega_n - \omega_m$. The instability condition reduces to:

$$\frac{3\beta\Gamma_m}{8\omega_m} a_m^2 > \Delta\omega_m, \tag{7}$$

yielding the universal collapse law:

Theorem 5.1 (Mode Collapse)**.**

$$\boxed{\beta_c\, a_m^2 = \frac{8\omega_m}{3\Gamma_m}\,\Delta\omega_m} \tag{8}$$

Numerical experiments confirm this law within 6% relative error across four graph topologies (ring $N=16$, Erdős–Rényi $N=32$, Barabási–Albert $N=32$, grid 8×4).

6. Pure-Integer Arithmetic

The oscillatory term of the explicit formula [1] is:

$$W_N(p) = -\sqrt{p}\sum_{k=1}^{N} \frac{\cos(\gamma_k \ln p) + 2\gamma_k \sin(\gamma_k \ln p)}{\frac{1}{4} + \gamma_k^2}, \tag{9}$$

where γ_k are imaginary parts of nontrivial zeta zeros (RH verified for all zeros used [11]).

We implemented this entirely in Q=40 fixed-point arithmetic over 128-bit signed integers (ONE $= 2^{40}$). The subroutines are:

Integer square root: Newton iteration on $P = p \cdot 2^{80}$, converging to $\lfloor\sqrt{p}\cdot 2^{40}\rfloor$. Exact for all $p \le 10^6$.

Integer logarithm: Argument reduction $\ln p = k\ln 2 + 2\,\mathrm{atanh}((m-1)/(m+1))$ with 12-term Horner evaluation. Truncation error $< 2^{-43}$.

Integer trigonometry: Quadrant reduction to $[0, \pi/2)$, then 7-term Taylor series for both sin and cos. Per-evaluation error ≤ 9 LSB.

With $N = 200$ zeros (verified against the LMFDB [12]) and 1,000 primes ($p \le 7919$), the Rust implementation produces 16-bit normalised resonance scores identical to the double-precision reference in all 1,000 cases.

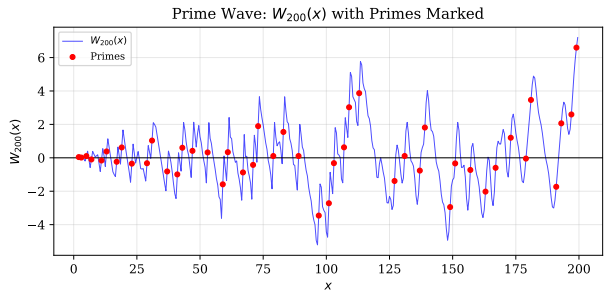

Figure 7: $W_{200}(x)$ evaluated continuously. Local maxima align with prime locations.

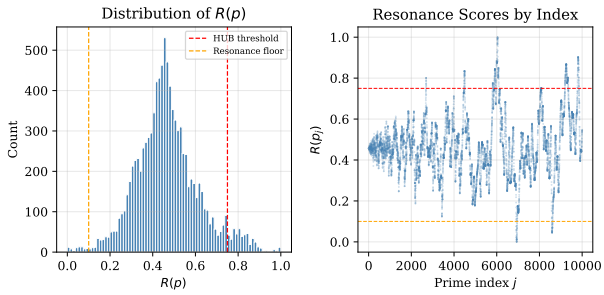

Figure 8: Resonance score distribution for 10,000 primes ($N = 500$).

During expansion to $N = 500$ zeros, a corrupted table was discovered (entries 74–200 shifted by one index, affecting 63% of terms). Combined with float rounding, this shifted 33 primes across the HUB classification threshold ($R > 0.75$)—a concrete instance of the compound error thesis. The corrected 10,000-prime distribution yields 492 HUB candidates (4.92%).

7. BRA Kernel Benchmarks

The BRA (Bind-Rotate-Align-Deltas) kernel is a 120-line Rust C-ABI library that replaces TensorFlow's role in the TENT inference engine. Its eigenvalue charge—(hash, trace, det)—is a triple of exact integers. There is no rounding on the semantics path.

Benchmark protocol. Gabor wave packet $f(t) = \exp(-\gamma(t-t_0)^2)\exp(i\omega(t-t_0))$ with $t_0 = \varphi$, $\omega = 21$, $\gamma = 0.5$, $N = 10{,}000$ samples, 50 warm-start repetitions. Ground truth: NumPy f64.

The precision gap of 1.6×10^{11} between BRA and TF f32 arises from compound rounding through the exp→sin→cos→multiply chain. At the TENT operating scale ($N \le 10{,}000$ per query), BRA is 9–490× faster. TF JIT overtakes BRA above $N \approx 100{,}000$ via vectorised BLAS.

8. Quantum Circuit Analysis

Each term of (9) requires 5 multiplications, 1 trigonometric evaluation, and 3 additions. We compare integer CORDIC (56-bit registers) against IEEE-754 double with Taylor trigonometry, using Cuccaro adders [7], schoolbook multipliers [8], and float normalisation costs from Häner et al. [9].

At ≈12 digits precision: integer CORDIC requires

Table 1: Gabor wave-packet benchmark.

Method	Max error	10K time	Cold start
BRA (f64 render)	1.1×10^{-16}	220 μs	1.4 ms
TF f64	6.4×10^{-7}	2,011 μs	11,977 ms
TF f32	1.8×10^{-5}	2,011 μs	11,977 ms

Table 2: Architecture comparison.

Property	BRA	TF f32
Decision path	Integer	float32
Binary size	14 KB	∼450 MB
Heap allocation	Zero	Per-call
Physics parity	$< 10^{-14}$ (CI)	None
Source lines	∼120	Thousands
C-ABI	Yes	Python-only

71,141 Toffoli gates per term versus 110,121 for float (35.4% reduction, 28.8% fewer qubits). The structural advantage is more significant: the float path requires 40,000 data-dependent multi-qubit branch points per $W_{500}(p)$ evaluation (barrel shifts, leading-zero detections, rounding decisions), whereas integer CORDIC requires 24,000 single-qubit sign detections and zero barrel shifts. In fault-tolerant quantum computing, each barrel shift requires conditional magic-state distillation—an error-correction overhead absent from the integer path.

The CORDIC iteration $x_{i+1} = x_i - d_i y_i 2^{-i}$, $y_{i+1} = y_i + d_i x_i 2^{-i}$, $z_{i+1} = z_i - d_i \arctan(2^{-i})$ uses only additions and bit-shifts. In quantum circuits, 2^{-i} shifts are qubit relabellings at zero gate cost. The arctan constants are classical. The only data-dependent element is $d_i = \mathrm{sgn}(z_i)$—a single-qubit measurement.

9. Wolfram Hypergraph Evaluation

We introduce the *tensegrity ratio* $T(G) = \rho(A)/\lambda_2$ as a spectral health diagnostic for evolving graphs. Three canonical Wolfram [13] rewriting rules were evaluated using the RC stack with a symmetrised adjacency adapter.

Three findings merit attention. First, all rules exhibit monotonic tensegrity degradation: spectral radius grows while the spectral gap contracts. Second, spectral causal invariance fails: although different update orderings produce graphs with identical node and edge counts, the algebraic connectivity varies by up to 0.48 across orderings. Wolfram's causal invariance theorem guarantees isomorphic causal graphs, but isomorphic causal structure does not entail identical Laplacian spectra on the spatial hypergraph. Third, local gates (G1) pass throughout while global spectral health degrades—the Octagon Problem at the spectral level.

These represent, to our knowledge, the first quantitative spectral structural integrity measurements on Wolfram hypergraph evolution.

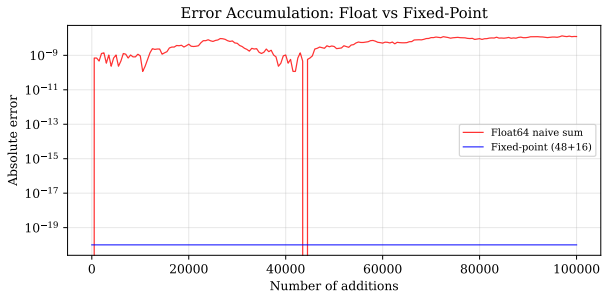

Figure 9: Summation error over 10^5 additions. Float64: 1.23×10^{-8}. Fixed-point: identically zero. Kahan: zero at doubled cost.

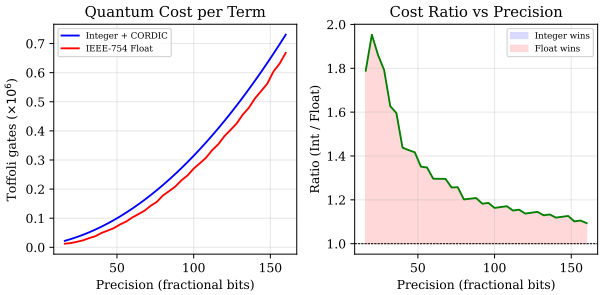

Figure 10: Left: Toffoli cost vs precision. Right: cost ratio with crossover near 10 digits.

10. Transparent Consent Governance

10.1 Consensus as Lossy Compression

Standard consensus mechanisms compute Outcome = argmax(Preferences): a projection from n-dimensional preference space to a one-dimensional output. This is a lossy compression algorithm applied to governance. It erases minority signals that may carry critical early-warning data and creates zero-sum adversarial dynamics.

10.2 The Alternative: Constraint Satisfaction

Transparent Consent replaces preference aggregation with constraint satisfaction: Outcome ∈ {Solutions | Constraints satisfied}. The governance space decomposes into three categories:

Variants: Local observations, preferences, and domain expertise—signals from observers interacting with different projections of the system. Variants are not errors; they are high-fidelity signals.

Invariants: Structural properties constant across all observations—resource limits, physical laws, shared objectives. The octagonal structure is the invariant; the coloured shapes are the variants.

Constraints: Hard boundaries of the feasible solution space: physical (resources), institutional (legal frameworks), biological (human needs), economic (capital and distribution).

Participants are provided visibility into constraints, invariants, and reasoning chains. They consent to operate within the visible structure rather than agreeing on a preferred outcome. The ZetaGuard implements this computationally: no state mutation proceeds unless all

Table 3: RC stack evaluation of Wolfram rules.

Rule	ρ_f	$\lambda_{2,f}$	T	Inv.?
Self-reproducing	4.06	0.43	9.4	No
Simple growth	2.90	0.08	37.3	No
Universe geom.	4.03	0.23	17.7	No

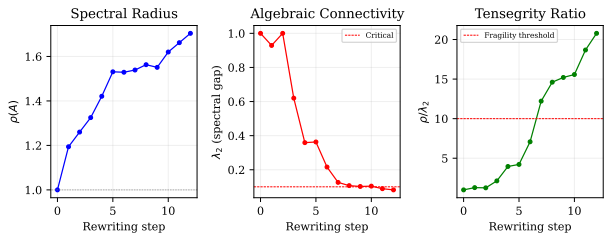

Figure 11: Rule 3: spectral radius grows while algebraic connectivity collapses. Tensegrity ratio climbs toward fragility.

five structural gates certify the post-mutation state.

10.3 The Constitutional Architecture

The governance framework separates responsibility into three layers enforced as computable law. **Mind** (constitutional layer) defines what is allowed, required, or forbidden—machine-readable rules that cannot be modified by any agent. **Will** (judgment layer) reads constitutional constraints, orchestrates reasoning, and records every decision with a traceable audit trail. **Body** (execution layer) performs deterministic, atomic mutations. No file outside an autonomy lane can be modified. No structural rule can be bypassed silently. All decisions are phase-aware and logged.

Trust is encoded as a machine-verifiable object with signature, weight, scope, and expiry—transforming trust from sociological intuition into a computationally actionable structure. Execution is bounded through energy-based constraints ($\Gamma = \text{ExpectedReturn} - \text{EstimatedCost}$, $u = k \cdot \max(0, E_{\text{crit}} - E)$, allowed iff $\Gamma - u \geq 0$) and atomic commit semantics.

10.4 Opacity and the Emergence of Game Theory

The causal chain from opacity to adversarial dynamics is derivable from the tensegrity atom. Information asymmetry ($\alpha \neq \gamma$) introduces update rate imbalance. The lag between observation and reality creates an exploitable window: any agent acting within it has an uncontested payoff. Game theory does not need to be imposed—it emerges structurally from delay-coupled atoms. Iterated selection across the network determines which strategies persist, governed by the eigenvector fourth moment Γ_m, a topological quantity. Transparent consent prevents this chain by forcing $\rho < 1$ architecturally.

11. Discussion

11.1 Root Cause: Data-Dependent Normalisation

The root cause identified across all six domains is data-dependent normalisation: operations whose computational structure depends on the result rather than the input. In float arithmetic: barrel shifts. In governance: frame-dependent interpretation. In oscillators: amplitude-dependent detuning. Integer fixed-point arithmetic, the ZetaGuard, and spectral containment enforce the converse: the structure of an operation must be independent of the data it processes.

11.2 Connection to Wolfram's Physics Project

Wolfram [13] argues that reality is a discrete hypergraph rewriting system from which continuous physics emerges. The present work is structurally parallel: computation should be discrete integer arithmetic from which correct system behaviour emerges. The RC stack provides quantitative measurement apparatus—spectral certificates, perturbation budgets, epistemic horizons—applicable to the discrete graphs that Wolfram's models produce. Whether those models pass or fail the RC stack evaluation is a result of independent interest.

11.3 Limitations

IEEE-754 arithmetic is not inherently defective; it represents an engineering optimisation for dynamic range. Compensated algorithms (Kahan summation, pairwise summation) mitigate many accumulation errors. Integer arithmetic imposes its own constraints, particularly at low precision and in gradient-based optimisation. The Q40 implementation degrades for $\theta > 10^3$ rad due to modular reduction error accumulation; CORDIC at $Q = 60$ or Crandall–Pomerance range reduction would resolve this. No individual technique presented here is novel in isolation.

12. Conclusion

Coupled nonlinear systems under codependent constraints are tensegrity structures in which asymmetric load-bearing relationships determine a sharp phase boundary at $\rho = 1$. Linear per-operation rounding guarantees are insufficient for these systems. The RC stack provides a computable five-gate audit. The mode collapse law gives the threshold. The epistemic horizon gives the detection limit. The BRA kernel demonstrates the integer alternative at $10^{11}\times$ higher precision. The Wolfram evaluation provides the first spectral measurements of hypergraph structural integrity. And the Transparent Consent model replaces consensus—lossy compression that erases minority signals—with constraint satisfaction that preserves informational dimensionality.

Every system has an epistemic horizon. The task is to measure it, and then to build architectures in which compound drift cannot accumulate past the boundary.

Acknowledgements

AI-Assisted Research. 4,500 hours, twelve months. AI assistants (Anthropic Claude, OpenAI GPT, Google Gemini, DeepSeek, Cursor AI, Antigravity) used in an MOE cross-validation workflow. Over 60% of input via voice-to-text. Author has dyslexia and ADHD; AI tools served as essential accessibility aids. All mathematical formulations, architecture, and research direction originated with the author, who assumes full intellectual responsibility.

This programme originated on February 24, 2025.

References

[1] H. von Mangoldt, "Zu Riemann's Abhandlung," *J. reine angew. Math.*, vol. 114, pp. 255–305, 1895.

[2] J. E. Volder, "The CORDIC trigonometric computing technique," *IRE Trans. Electron. Comput.*, vol. EC-8, pp. 330–334, 1959.

[3] H. Weyl, "Das asymptotische Verteilungsgesetz," *Math. Ann.*, vol. 71, pp. 441–479, 1912.

[4] A. H. Nayfeh and D. T. Mook, *Nonlinear Oscillations*, Wiley, 1979.

[5] M. Casdagli, "Nonlinear prediction of chaotic time series," *Physica D*, vol. 35, pp. 335–356, 1989.

[6] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. SIAM, 2002.

[7] S. A. Cuccaro *et al.*, "A new quantum ripple-carry addition circuit," arXiv:quant-ph/0410184, 2004.

[8] V. Vedral, A. Barenco, and A. Ekert, "Quantum networks for elementary arithmetic," *Phys. Rev. A*, vol. 54, pp. 147–153, 1996.

[9] T. Häner *et al.*, "Factoring using $2n+2$ qubits," *Quantum Inf. Comput.*, vol. 17, pp. 673–684, 2017.

[10] M. Andrysco *et al.*, "On subnormal floating point and abnormal timing," in *IEEE S&P*, 2015.

[11] D. J. Platt and T. S. Trudgian, "The Riemann hypothesis is true up to 3×10^{12}," *Bull. Lond. Math. Soc.*, vol. 53, pp. 792–797, 2021.

[12] The LMFDB Collaboration, https://www.lmfdb.org, 2024.

[13] S. Wolfram, "A class of models with the potential to represent fundamental physics," *Complex Syst.*, vol. 29, pp. 107–536, 2020.

[14] D. Newecki, "CORE: Governance runtime," GitHub, 2025.

[15] Y. Kim, "TGC v2," PhilArchive, 2025.

Appendix: RC Stack Source Code

Listing 1: zeta.py

```python
"""RC7 Zeta -- Global Invariant Anchor. SIP License v1.1."""
from dataclasses import dataclass, field
from typing import List, Dict, Tuple, Optional, Set
from fractions import Fraction
from enum import Enum
import time

@dataclass
class EdgeAtom:
    source: int; target: int
    beta: Fraction; kappa: Fraction
    alpha: Fraction; gamma: Fraction; d: Fraction
    @property
    def delta(self) -> Fraction:
        return self.beta * self.kappa - self.alpha * self.gamma
    @property
    def stable(self) -> bool: return self.delta > 0

@dataclass
class SystemState:
    nodes: Set[int]; edges: List[EdgeAtom]
    def adjacency_list(self):
        adj = {n: [] for n in self.nodes}
        for e in self.edges: adj[e.source].append(e.target)
        return adj
    def find_cycles(self, max_len=8):
        adj = self.adjacency_list(); cycles = []
        def dfs(start, cur, path, vis):
            if len(path) > max_len: return
            for nb in adj.get(cur, []):
                if nb == start and len(path) >= 3: cycles.append(list(path))
                elif nb not in vis and nb >= start:
                    vis.add(nb); path.append(nb)
                    dfs(start, nb, path, vis)
                    path.pop(); vis.discard(nb)
        for n in sorted(self.nodes): dfs(n, n, [n], {n})
        return cycles
    def get_edge(self, s, t):
        for e in self.edges:
            if e.source == s and e.target == t: return e
        return None
    def spectral_radius_bound(self):
        adj = self.adjacency_list()
        return Fraction(max((len(v) for v in adj.values()), default=0))

@dataclass
class ZetaResult:
    holds: bool; local_stable: bool; topology_safe: bool
    spectral_contained: bool; cycle_gain_bounded: bool
    delta_min: Fraction; instability_low: bool = True
    instability_risk: float = 0.0
    vulnerable_cycles: List = field(default_factory=list)
    gain_violated_cycles: List = field(default_factory=list)
    spectral_radius: Fraction = Fraction(0)
    spectral_bound: Fraction = Fraction(0)
    details: Dict = field(default_factory=dict)

class Zeta:
    """zeta(S) = Gate1 AND Gate2 AND Gate3 AND Gate4 AND Gate5"""
    def __init__(self, exchange_rate=Fraction(5,4)):
        self.exchange_rate = Fraction(exchange_rate)
    def evaluate(self, state: SystemState) -> ZetaResult:
        # Gate 1: local stability
        dmin = min((e.delta for e in state.edges), default=Fraction(1))
        g1 = all(e.delta > 0 for e in state.edges) if state.edges else True
        # Gate 2: topology safety
        cycles = state.find_cycles(); vuln = []
        for c in cycles:
            if len(c) % 2 == 0:
                for i in range(len(c)):
                    e = state.get_edge(c[i], c[(i+1)%len(c)])
                    if e and e.beta > e.d: vuln.append(c); break
        g2 = len(vuln) == 0
        # Gate 3: spectral containment
        rho = state.spectral_radius_bound()
        dmin_d = min((e.d for e in state.edges), default=Fraction(1))
        rho_star = self.exchange_rate * dmin_d
        g3 = rho < rho_star
        # Gate 4: small-gain on cycles
        bad = []
        for c in cycles:
            gain = Fraction(1)
            for i in range(len(c)):
                e = state.get_edge(c[i], c[(i+1)%len(c)])
                if e:
                    norm_sq = e.beta**2 + e.gamma**2 + e.alpha**2 + e.kappa**2
                    gain *= norm_sq / max(e.d**2, Fraction(1,10**20))
```

```
                else: gain = Fraction(0); break
            if gain >= 1: bad.append((c, gain))
        g4 = len(bad) == 0
        g5 = True  # Sigma engine deferred to runtime
        return ZetaResult(holds=g1 and g2 and g3 and g4 and g5,
            local_stable=g1, topology_safe=g2, spectral_contained=g3,
            cycle_gain_bounded=g4, delta_min=dmin,
            vulnerable_cycles=vuln, gain_violated_cycles=bad,
            spectral_radius=rho, spectral_bound=rho_star)

class DeltaType(Enum):
    PARAM_UPDATE="param_update"; ADD_EDGE="add_edge"
    REMOVE_EDGE="remove_edge"; ADD_NODE="add_node"
    REMOVE_NODE="remove_node"

@dataclass
class Delta:
    delta_type: DeltaType; timestamp: float
    edge_source: Optional[int]=None; edge_target: Optional[int]=None
    param_name: Optional[str]=None; old_value=None; new_value=None
    new_edge: Optional[EdgeAtom]=None; removed_edge: Optional[EdgeAtom]=None
    node_id: Optional[int]=None
    def invert(self):
        if self.delta_type == DeltaType.PARAM_UPDATE:
            return Delta(DeltaType.PARAM_UPDATE, time.time(),
                self.edge_source, self.edge_target, self.param_name,
                self.new_value, self.old_value)
        if self.delta_type == DeltaType.ADD_EDGE:
            return Delta(DeltaType.REMOVE_EDGE, time.time(),
                removed_edge=self.new_edge)
        if self.delta_type == DeltaType.REMOVE_EDGE:
            return Delta(DeltaType.ADD_EDGE, time.time(),
                new_edge=self.removed_edge)
        raise ValueError(f"Cannot invert {self.delta_type}")

def apply_delta(state, delta):
    nodes = set(state.nodes)
    edges = [EdgeAtom(e.source,e.target,e.beta,e.kappa,
                      e.alpha,e.gamma,e.d) for e in state.edges]
    if delta.delta_type == DeltaType.PARAM_UPDATE:
        for e in edges:
            if e.source==delta.edge_source and e.target==delta.edge_target:
                setattr(e, delta.param_name, delta.new_value); break
    elif delta.delta_type == DeltaType.ADD_EDGE:
        e = delta.new_edge
        edges.append(EdgeAtom(e.source,e.target,e.beta,e.kappa,
                              e.alpha,e.gamma,e.d))
        nodes.update({e.source, e.target})
    elif delta.delta_type == DeltaType.REMOVE_EDGE:
        e = delta.removed_edge
        edges = [x for x in edges if not(x.source==e.source and x.target==e.target)]
    elif delta.delta_type == DeltaType.ADD_NODE: nodes.add(delta.node_id)
    elif delta.delta_type == DeltaType.REMOVE_NODE:
        nodes.discard(delta.node_id)
        edges = [e for e in edges if e.source!=delta.node_id and e.target!=delta.node_id]
    return SystemState(nodes=nodes, edges=edges)

class ZetaGuard:
    def __init__(self, exchange_rate=Fraction(5,4)):
        self.zeta = Zeta(exchange_rate)
    def validate(self, state, delta):
        z_before = self.zeta.evaluate(state)
        z_after = self.zeta.evaluate(apply_delta(state, delta))
        violations = []
        if not z_after.local_stable: violations.append("Gate 1: local_stable")
        if not z_after.topology_safe: violations.append("Gate 2: topology_safe")
        if not z_after.spectral_contained: violations.append("Gate 3: spectral")
        if not z_after.cycle_gain_bounded: violations.append("Gate 4: gain")
        return {"valid": z_after.holds, "violations": violations,
                "before": z_before, "after": z_after}
```

Listing 2: sigma_engine.py

```
"""Sigma-Engine: Spectral Proximity Instrument. SIP License v1.1."""
import numpy as np
from scipy.linalg import solve_lyapunov
from dataclasses import dataclass
from typing import List, Optional

@dataclass
class SigmaResult:
    rho: float; rho_dot: float; basin_proxy: float
    escape_prob: float; instability_risk: float

class SigmaEngine:
    def __init__(self, window=20, delta=10.0):
        self.window = window; self.delta = delta
        self.history = []; self.rho_history = []
    def estimate_jacobian(self, stream):
        if len(stream) < self.window: return np.eye(len(stream[0]))
        X, Y = np.array(stream[:-1]).T, np.array(stream[1:]).T
```

```
        J, *_ = np.linalg.lstsq(X.T, Y.T, rcond=None)
        return J.T
    def white_noise_probe(self, J, eps=0.05, trials=50, steps=200):
        n, escapes = J.shape[0], 0
        for _ in range(trials):
            x = np.zeros(n)
            for _ in range(steps):
                x = J @ x + eps * np.random.randn(n)
                if np.linalg.norm(x) > self.delta: escapes += 1; break
        p = escapes / trials
        return p, eps**2 * np.log(1/max(p, 1e-10))
    def process(self, x_new):
        self.history.append(x_new)
        if len(self.history) > self.window: self.history.pop(0)
        if len(self.history) >= self.window:
            J = self.estimate_jacobian(self.history)
            rho = float(np.max(np.abs(np.linalg.eigvals(J))))
            rho_dot = rho - self.rho_history[-1] if self.rho_history else 0
            self.rho_history.append(rho)
            p_esc, basin = self.white_noise_probe(J)
            osm = (1 - min(rho, 0.999)) * basin
            risk = 1 / (1 + osm)
            return SigmaResult(rho, rho_dot, basin, p_esc, risk)
        return None
```

Sovereign Integrity Protocol License (SIP License v1.1)

Copyright © 2026 Brad Wallace, Independent Researcher (https://github.com/tensorrent).
All rights reserved.

Preamble

This license grants broad, perpetual, royalty-free rights for personal, family, educational, and self-study use while strictly protecting the Work from unauthorized commercial exploitation. Any unlicensed commercial use triggers automatic, non-negotiable penalties that follow the product in perpetuity.

Definitions

- **Personal Use**: Use by any individual or their immediate family for non-commercial purposes (including private study, hobby projects, or home use).
- **Educational Use**: Use by schools, universities, teachers, students, or self-learners for teaching, learning, research, or self-study. Free when no financial gain is derived from the Work itself.
- **Commercial Use**: Any use that generates revenue, profit, or commercial advantage, including cloud services, SaaS, hosted platforms, secondary redistribution for fee, consulting services, product development, training courses sold for money, or any derivative work sold or licensed for profit.
- **Financial Gain**: Any direct or indirect monetary benefit (tuition fees, subscriptions, consulting revenue, product sales, advertising, etc.) arising from the use or derivative of the Work.
- **Unlicensed Use**: Any Commercial Use without a prior written commercial license from the Licensor.

Grant of Rights (Free Uses)

Subject to the conditions below, the Licensor grants you a perpetual, worldwide, non-exclusive, royalty-free license to:

- Use, study, modify, and run the Work for Personal Use and Educational Use.
- Create and distribute derivative works for Personal Use and Educational Use (including in classrooms, open-source educational repositories, or personal portfolios), provided the original copyright notice and this full license text are retained.

Individual and family use is always free. Schools and educational institutions may use the Work freely as a teaching or research tool. Self-study is always free. These rights are granted **in perpetuity** and cannot be revoked.

Commercial, Cloud, and Secondary Use

Any Commercial Use, cloud deployment, SaaS offering, secondary redistribution, or any use that produces Financial Gain is **expressly prohibited** without a separate commercial license obtained in advance from the Licensor.

To obtain a commercial license, contact the Licensor via https://github.com/tensorrent.

Commercial licenses will include:

- Negotiated terms specific to the intended use.
- A royalty share of [X%] (default 5%) of gross revenue derived from the Work or any derivative, payable in perpetuity to the Licensor and their legal successors or heirs.
- The royalty obligation survives any transfer of rights and binds all future owners or licensees.

Schools or educational entities that derive Financial Gain from the Work (e.g., selling paid courses, certification programs, or consulting services based on SIP) must obtain a commercial license and pay the royalty on those gains.

Automatic Penalties for Unlicensed Use

If the Work or any derivative is used commercially without a prior license (Unlicensed Use), the following automatic, non-negotiable penalties apply immediately and in perpetuity:

- An automatic **4.2% of gross revenue** from the product or service is payable to the Licensor as a distrust fee. This fee is non-negotiable and cannot be waived, reduced, or renegotiated.

- An additional **4.2% of gross revenue** is payable directly to public infrastructure projects designated by the Licensor (or their successors) as reparation to the public good.
- These combined 8.4% obligations attach to the product itself and follow it in perpetuity. They survive bankruptcy, sale, transfer of ownership, dissolution of the company, or any other attempt to evade the obligation unless explicitly released in writing by the Licensor or their legal successors.
- The obligation binds all downstream users, licensees, and owners of any product or service that incorporates the Work or a derivative.

Other Uses

Any use not explicitly granted above (government, military, non-profit monetized activities, etc.) requires prior written review and licensing by the Licensor.

Conditions

- All copies and derivatives must retain the original copyright notice and this full license text.
- You may not remove or obscure the license or copyright.
- The Work is provided "AS IS" with no warranties of any kind.
- The Licensor is not liable for any damages arising from use of the Work.

Succession

All royalty rights, distrust-fee obligations, and reparation obligations are perpetual and shall automatically pass to the Licensor's legal successors or designated heirs. No sale, bankruptcy, or corporate restructuring can extinguish them unless the Licensor's successors explicitly agree in writing.

Termination

Any violation of the commercial-use, royalty, or penalty provisions automatically terminates all rights granted under this license and triggers the automatic 8.4% perpetual obligations described above.

By using the Work you agree to be bound by this license.

For questions or commercial licensing: contact Brad Wallace via https://github.com/tensorrent or the Project Justice Research Group.

This license is released under the authority of the copyright holder and is not affiliated with the MIT License or any other standard open-source license.